

SOFTWARE VERIFICATION AND VALIDATION

Project report

Title:

Complaint Management System (Ticket-Based Workflow)



Submitted to:

Ma'am Javeria Ramzan

Submitted by:

Alisha Kanwal (FA-23/BSSE/165-D)

Alina Irshad (FA-23/BSSE/199-D)

Mubina (FA-23/BSSE/193-D)

Session 2023-2027

Department of Software Engineering

Lahore Garrison University

Contents

1. INTRODUCTION.....	4
2. PROJECT OBJECTIVES	4
3. PROJECT SCOPE.....	4
3.1 Included Features.....	4
3.2 Excluded Features	5
4. METHODOLOGY.....	5
Phase 1 – Requirement Engineering.....	5
Phase 2 – Formal Modeling (Z Notation)	5
Phase 3 – Functional Specification (VDM).....	6
Phase 4 – Structural Verification (Alloy)	6
Phase 5 – Validation and Security	6
5. REQUIREMENT ENGINEERING	6
5.1 Software Requirements Specification (SRS).....	6
5.2 Requirement Defect Analysis	7
5.3 GitHub Issues Tracking.....	7
6. FORMAL MODELING USING Z NOTATION.....	8
6.1 System Components	8
6.2 System Invariants.....	8
6.3 Operations Defined.....	8
7. FUNCTIONAL SPECIFICATION USING VDM.....	10
7.1 Verified Behaviors	10
7.2 Example Conditions	10
8. STRUCTURAL VERIFICATION USING ALLOY.....	11
8.1 Alloy Model Components.....	11
8.2 Constraint Verification	11
9. VALIDATION AND SECURITY	13
9.1 Validation Checklist	13
9.2 Negative Testing	13
9.3 Role-Based Validation	13
10. CI/CD PIPELINE EVIDENCE	14
11. TOOLS USED	15
12. RESULTS AND FINDINGS	15

13.	CONCLUSION	15
14.	REFERENCES	16
15.	GitHub Link	16

1.INTRODUCTION

The Complaint Management System is a ticket-based workflow system designed to manage complaints in an organized and structured manner.

In many organizations, complaints are handled manually, which may cause delays, missing records, and improper tracking. The system includes three major roles:

- User
- Support Agent
- Admin

Each complaint follows a lifecycle:

Open → Assigned → In Progress → Resolved → Closed → Reopened

The main purpose of this project is not implementation or coding, but the formal verification and validation of system behavior using formal methods and verification tools.

2.PROJECT OBJECTIVES

The main objectives of the project are:

- To design a structured complaint management workflow
- To ensure ticket lifecycle correctness
- To validate role-based operations
- To maintain system consistency using invariants
- To detect possible logical errors using formal verification tools
- To apply Software Verification and Validation (SVV) techniques in a real-world problem

3.PROJECT SCOPE

The system manages complaints from ticket creation to final closure.

3.1 Included Features

- Ticket creation
- Ticket assignment
- Complaint processing

- Status tracking
- Complaint resolution
- Ticket closure
- Ticket reopening

3.2 Excluded Features

- Payment systems
- Email/SMS notifications
- Third-party integrations
- Mobile application support

4.METHODOLOGY

The project follows the Software Verification and Validation (SVV) pipeline defined in the lab guidelines.

Phase 1 – Requirement Engineering

Requirements were collected and documented in the Software Requirements Specification (SRS).Activities performed:

- Functional requirement analysis
- Non-functional requirement analysis
- Requirement defect identification
- GitHub issue tracking

Phase 2 – Formal Modeling (Z Notation)

The system behavior and states were formally modeled using Z Notation. The following were defined:

- System states
- Invariants
- Operations
- State transitions

Phase 3 – Functional Specification (VDM)

VDM specifications were used to define:

- Preconditions
- Postconditions
- Functional behavior of operations

Phase 4 – Structural Verification (Alloy)

The Alloy Analyzer was used for relational modeling and constraint verification. The following were verified:

- Ticket relationships
- Status constraints
- Workflow correctness
- Invariant checking
- Counterexample generation

Phase 5 – Validation and Security

The final phase included:

- Validation checklist
- Requirement traceability
- CI/CD pipeline verification
- GitHub Actions validation

5.REQUIREMENT ENGINEERING

5.1 Software Requirements Specification (SRS)

The SRS document defines:

- System purpose
- Scope
- User roles
- Functional requirements
- Non-functional requirements
- Workflow rules
- System invariants

Functional Requirements

ID	Requirement
FR1	Create Ticket
FR2	Assign Ticket
FR3	Start Processing
FR4	Resolve Ticket
FR5	Close Ticket
FR6	Reopen Ticket

5.2 Requirement Defect Analysis

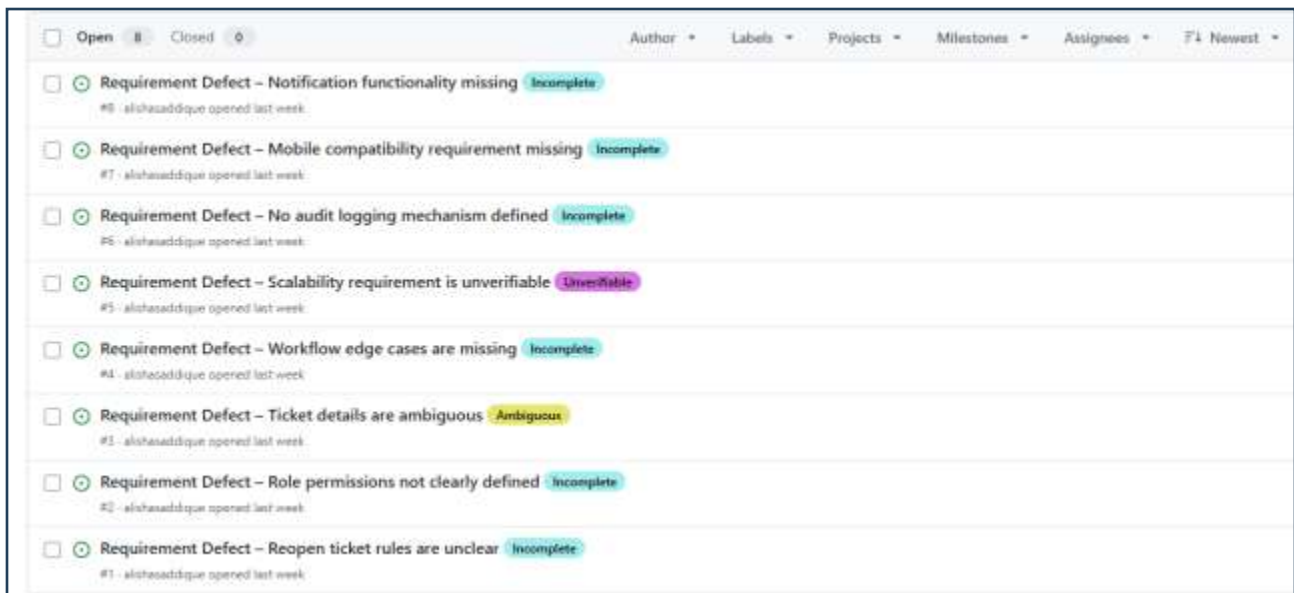
Requirement defects were identified during requirement verification.

Defect ID	Type	Description
D1	Ambiguous	Ticket fields not clearly defined
D2	Incomplete	Reopen conditions unclear
D3	Missing	Role permissions incomplete

5.3 GitHub Issues Tracking

GitHub Issues were used to track requirement defects and documentation problems. Examples of Created Issues

- Requirement ambiguous – Ticket details unclear
- Reopen ticket workflow incomplete
- Role-based permissions not fully specified



The screenshot displays a GitHub Issues page with a list of eight issues. Each issue is titled 'Requirement Defect - [description]' and is labeled with a status tag. The issues are as follows:

Issue Title	Status
Requirement Defect – Notification functionality missing	Incomplete
Requirement Defect – Mobile compatibility requirement missing	Incomplete
Requirement Defect – No audit logging mechanism defined	Incomplete
Requirement Defect – Scalability requirement is unverifiable	Unverifiable
Requirement Defect – Workflow edge cases are missing	Incomplete
Requirement Defect – Ticket details are ambiguous	Ambiguous
Requirement Defect – Role permissions not clearly defined	Incomplete
Requirement Defect – Reopen ticket rules are unclear	Incomplete

Figure 1 Github Issues

6.FORMAL MODELING USING Z NOTATION

Z Notation was used to mathematically define the Complaint Management System.

6.1 System Components

- Tickets
- Ticket Status
- Assigned Agent
- Resolution Notes

6.2 System Invariants

The following invariants were verified:

- Every ticket must have a unique ID
- Closed tickets cannot be reassigned
- Resolution notes are required for resolved tickets

6.3 Operations Defined

- Create Ticket
- Assign Ticket
- Start Processing
- Resolve Ticket
- Close Ticket

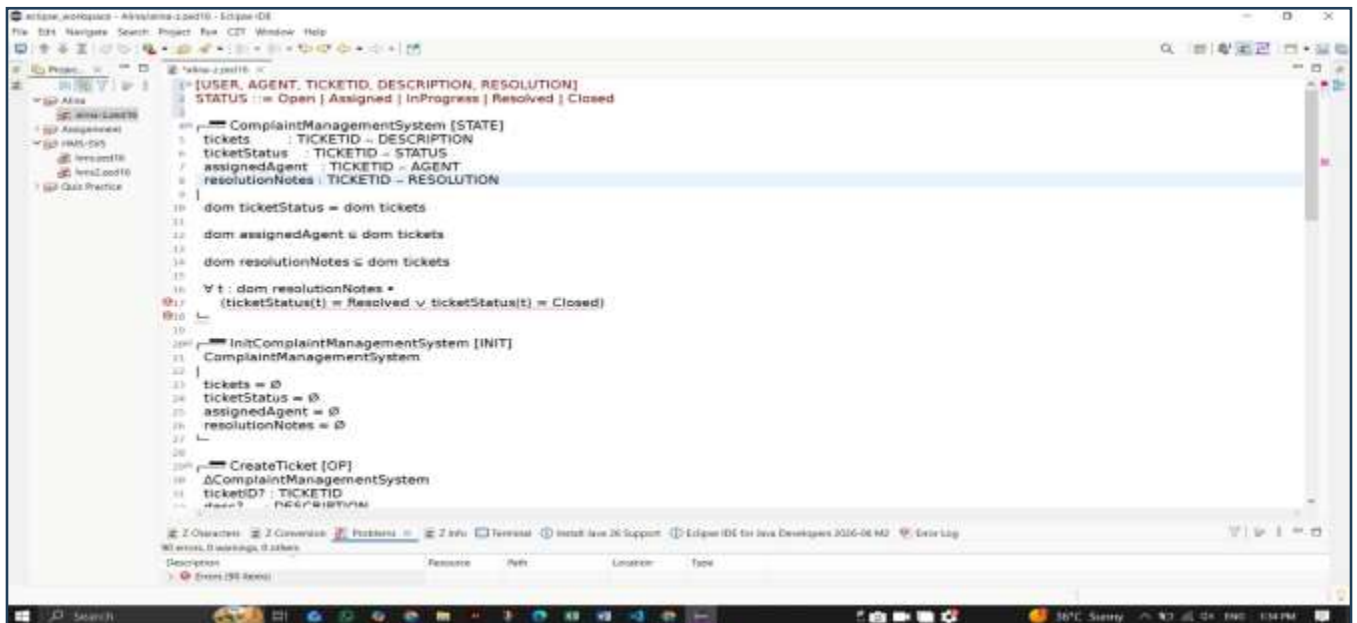


Figure 2 Assign Ticket and Start Processing Operations in Z Notation


```

1 := {USER, AGENT, TICKETID, DESCRIPTION, RESOLUTION}
2 STATUS ::= Open | Assigned | InProgress | Resolved | Closed
3
4 == ComplaintManagementSystem [STATE]
5 tickets : TICKETID -> DESCRIPTION
6 ticketStatus : TICKETID -> STATUS
7 assignedAgent : TICKETID -> AGENT
8 resolutionNotes : TICKETID -> RESOLUTION
9 |
10 dom ticketStatus = dom tickets
11
12 dom assignedAgent ⊆ dom tickets
13
14 dom resolutionNotes ⊆ dom tickets
15
16 ∀ t : dom resolutionNotes •
17 (ticketStatus(t) = Resolved ∨ ticketStatus(t) = Closed)
18
19
20 == InitComplaintManagementSystem [INIT]
21 ComplaintManagementSystem
22 |
23 tickets = ∅
24 ticketStatus = ∅
25 assignedAgent = ∅
26 resolutionNotes = ∅
27

```

Figure 3 System State, Invariants, and Initial State in Z Notation

```

10
11 ticketStatus' =
12   ticketStatus ∪ {ticketID? ↦ Open}
13
14 assignedAgent' = assignedAgent
15 resolutionNotes' = resolutionNotes
16
17
18 == AssignTicket [OP]
19 ΔComplaintManagementSystem
20 ticketID? : TICKETID
21 agent? : AGENT
22 |
23 ticketID? ∈ dom tickets
24
25 ticketStatus(ticketID?) = Open
26
27 assignedAgent' =
28   assignedAgent ∪ {ticketID? ↦ agent?}
29
30 ticketStatus' =
31   ticketStatus ∪ {ticketID? ↦ Assigned}
32
33 tickets' = tickets
34 resolutionNotes' = resolutionNotes
35
36
37 == StartProcessing [OP]
38 ΔComplaintManagementSystem
39 ticketID? : TICKETID
40 |

```

Z Characters Z Conversion Z Info Terminal Install Java 26 Support Eclipse IDE for Java Developers 2026-06 M2

The 2026-06 M2 milestone for the Eclipse IDE for Java Developers is available. It provides early access to the latest and greatest features for your IDE, though of course there is also the possibility of regressions.

Figure 4 Complete Z Specification of Complaint Management System with CZT Tool Output

7.FUNCTIONAL SPECIFICATION USING VDM

VDM was used for functional specification and operation verification.

7.1 Verified Behaviors

- Ticket creation rules
- Status transition correctness
- Preconditions and postconditions
- Workflow consistency

7.2 Example Conditions

Preconditions

- Ticket must exist before assignment
- Resolution required before closing

Postconditions

- Ticket status updates correctly
- Resolution notes stored successfully

```
**
** Welcome to the Overture Interactive Console
**
> init
Cleared all coverage information
Global context initialized
> print CreateTicket("Cannot access database", 10)
()
> print tickets
{1 |-> mk_Ticket(1, "Cannot access database", <Open>, 10, nil, nil)}
> print AssignTicket(1, 99)
()
> print StartProcessing(1, 99)
()
> print ResolveTicket(1, "Restarted the database service.")
()
> print CloseTicket(1)
()
> print tickets
{1 |-> mk_Ticket(1, "Cannot access database", <Closed>, 10, 99, "Restarted the database service.")}
>
```

Figure 5 VDM

8.STRUCTURAL VERIFICATION USING ALLOY

Alloy Analyzer was used to verify relational constraints and workflow correctness.

8.1 Alloy Model Components

Signatures

- User
- Admin
- SupportAgent
- Ticket
- Status
- Note

Relations

- assignedTo
- status
- resolution

8.2 Constraint Verification

The following assertions were checked:

Assertion 1 – NoDuplicateIDs

Purpose:

Verify that every ticket has a unique ID.

Result:

No counterexample found.

Assertion 2 – ClosedTicketNoAgent

Purpose:

Verify that closed tickets cannot be reassigned.

Result:

No counterexample found.

Assertion 3 – WrongRule (Counterexample)

Purpose:

Intentionally test an incorrect rule.

Result:

Counterexample generated successfully.

Analysis:

The Alloy Analyzer detected a violation because the intentionally incorrect assertion allowed invalid ticket behavior. This demonstrates the effectiveness of Alloy in detecting logical inconsistencies.

```
Executing "Check NoDuplicateIDs for 5"
Actual scopes: 5 User, 5 Customer, 5 SupportAgent, 5 Admin, 5 Status, exactly 1 Open, exactly 1 Assigned, exactly 1 InProgress, exactly 1 Resolved, exactly 1 Closed, 5 Note, 5 Ticket
Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
2205 vars, 190 primary vars, 4245 clauses, 201ms.
No counterexample found. Assertion may be valid. 44ms.
```

Figure 6 Check noduplicationids for 5

```
Executing "Check ClosedTicketNoAgent for 5"
Actual scopes: 5 User, 5 Customer, 5 SupportAgent, 5 Admin, 5 Status, exactly 1 Open, exactly 1 Assigned, exactly 1 InProgress, exactly 1 Resolved, exactly 1 Closed, 5 Note, 5 Ticket
Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
1999 vars, 185 primary vars, 3551 clauses, 77ms.
No counterexample found. Assertion may be valid. 10ms.
```

Figure 7 check closedticketnoagent for 5

```
Executing "Run run$3 for 5"
Actual scopes: 5 User, 5 Customer, 5 SupportAgent, 5 Admin, 5 Status, exactly 1 Open, exactly 1 Assigned, exactly 1 InProgress, exactly 1 Resolved, exactly 1 Closed, 5 Note, 5 Ticket
Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
1897 vars, 180 primary vars, 3374 clauses, 92ms.
Instance found. Predicate is consistent. 42ms.
```

Figure 9 run for 5 instance found

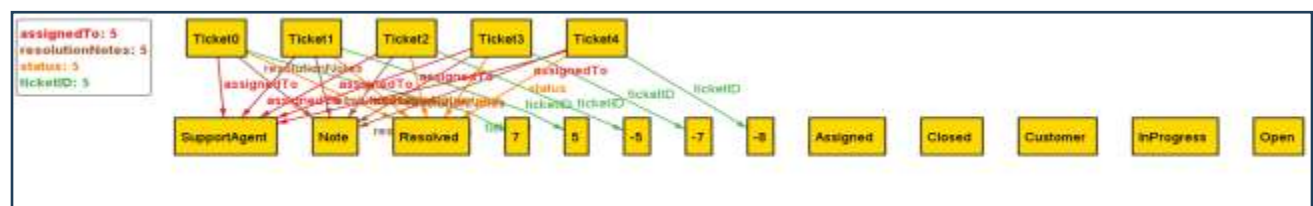


Figure 8 instance found

Executing "Check WrongRule for 5"

Actual scopes: 5 User, 5 Customer, 5 SupportAgent, 5 Admin, 5 Status, exactly 1 Open, exactly 1 Assigned, exactly 1 InProgress, exactly 1 Resolved, exactly 1 Closed, 5 Note, 5 Ticket
Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
1999 vars, 185 primary vars, 3550 clauses, 79ms.
Counterexample found. Assertion is invalid. 27ms.

Figure 10 counter example found

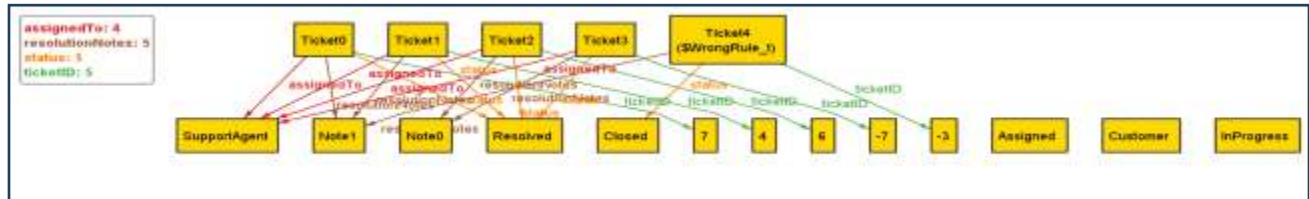


Figure 11 graph of counter example

9.VALIDATION AND SECURITY

9.1 Validation Checklist

Requirement	Status
Create Ticket	Pass
Assign Ticket	Pass
Resolve Ticket	Pass
Close Ticket	Pass
Reopen Ticket	Pass

9.2 Negative Testing

Test Case	Expected Result
Empty complaint	Rejected
Close without resolution	Blocked
Unauthorized access	Denied
Modify closed ticket	Prevented

9.3 Role-Based Validation

Function	User	Admin	Agent
Create Ticket	✓	✗	✗

Assign Ticket	X	✓	X
Resolve Ticket	X	X	✓
Close Ticket	X	✓	X

10. CI/CD PIPELINE EVIDENCE

GitHub Actions was used to validate repository workflow automatically. The CI pipeline successfully executed without errors.

Validation included:

- Repository structure verification
- Workflow execution
- Automatic pipeline success check



Figure 12 Workflow YAML File

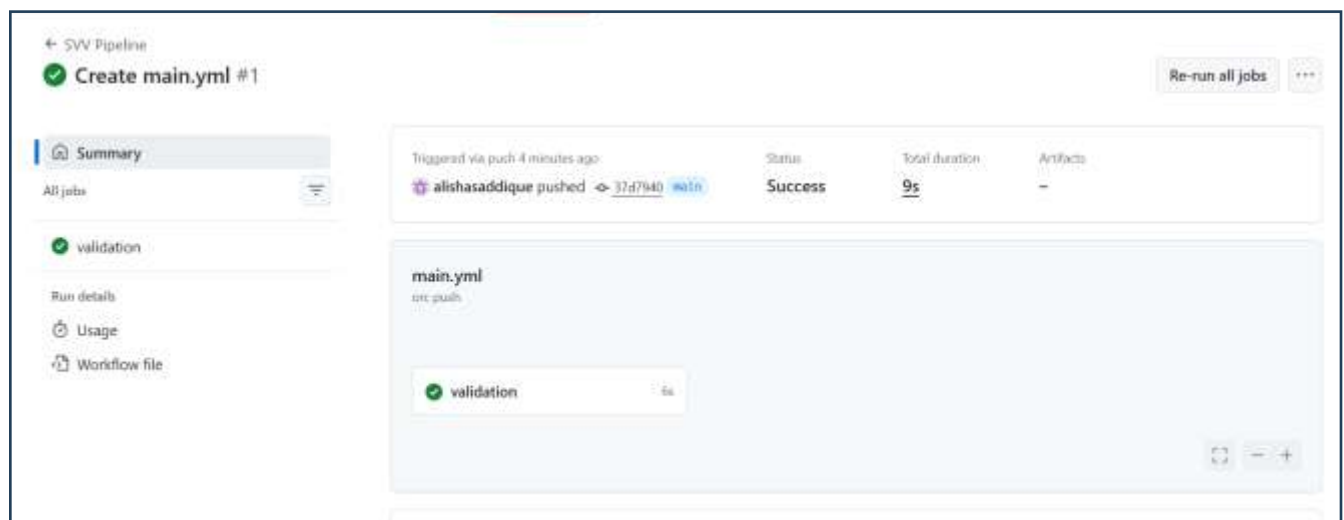


Figure 13 GitHub Actions Success

11. TOOLS USED

Tool	Purpose
GitHub	Repository and issue tracking
GitHub Actions	CI/CD validation
CZT	Z Notation verification
VDMTools / Overture	VDM specification
Alloy Analyzer	Structural verification
Microsoft Word	Documentation

12. RESULTS AND FINDINGS

The project successfully verified the Complaint Management System using formal methods.

Major Findings

- All workflow transitions were valid
- System invariants remained consistent
- Role-based constraints were enforced
- Alloy successfully detected incorrect rules
- Formal verification improved system reliability

13. CONCLUSION

The Complaint Management System was successfully modeled, verified, and validated using formal verification techniques.

The project demonstrated how formal methods can improve software correctness and system reliability.

All required phases of the SVV pipeline were completed successfully, including:

- Requirement Engineering
- Z Modeling
- VDM Specification
- Alloy Verification
- Validation and CI/CD

The final system satisfies all defined functional requirements and maintains consistency across all formal models.

14. REFERENCES

- Software Verification and Validation Lab Manual
- Alloy Analyzer Documentation
- CZT Z Notation Tool Documentation
- VDMTools / Overture Documentation
- GitHub Documentation

15. GitHub Link

<https://github.com/alishasaddique/complaint-management-system-svv.git>